

СОДЕРЖАНИЕ

Введение.....	5
1 Описание предприятия	6
1.1 Структура предприятия и описание его бизнес-процессов	6
1.2 Организация экономической работы	6
1.3 Индивидуальное задание.....	7
2 Обзор литературы	8
2.1 Java.....	8
2.2 PostgreSQL	9
2.3 Spring Framework.....	11
2.4 Maven.....	12
2.5 Инструменты разработки: IntelliJ IDEA	13
2.6 Инструменты аутентификации: JWT (JSON Web Token).....	14
2.7 Инструменты управления версиями: Git	15
3 Выполнение индивидуального задания	16
3.1 Архитектура и организация проекта	16
3.2 Работа с базой данных и миграциями	16
3.3 Разработка REST API и бизнес-логики.....	17
3.4 Обеспечение безопасности: Spring Security и JWT	17
3.5 Среда разработки и инструменты автоматизации	18
3.6 Документирование и поддержка API.....	18
Заключение	19
Список использованных источников	20

ВВЕДЕНИЕ

Производственная практика – это часть учебного процесса, необходимая для подготовки квалифицированных работников путем формирования у них практических навыков.

Производственная практика предоставляет возможность закрепить, проверить и обобщить знания, полученные в процессе обучения, путем применения их на практике, осознать сложность профессии и понять специфику обязанностей, которые будут возложены на нее, возможность получения навыков в поиске работы и проведения переговоров с работодателем, возможность дальнейшего трудоустройства в компании, где проходила производственная практика.

Цель производственной практики: проверка полученных в процессе обучения знаний, а также приобретение профессиональных навыков необходимых для работы по специальности.

Работа в ООО «Модсен» позволяет не только применить теоретические знания на практике, но и развивать навыки взаимодействия с командой разработчиков, что является важным опытом для понимания процессов коллективной работы в IT-сфере. Участие в реальных проектах способствует формированию профессионального подхода к решению задач и пониманию важности соблюдения сроков и стандартов качества.

В ходе поиска работы местом прохождения практики стало ООО «Модсен». Прохождение производственной практики ставит следующие задачи:

- ознакомление со структурой предприятия;
- изучение технологического процесса или описание бизнес-процесса, происходящего на предприятии;
- ознакомление с основами проектирования программных средств или их составных частей;
- ознакомление с основами работы по стандартизации на предприятии;
- сбор материала для формирования технических задач, которую планируется решать в дипломном проекте;
- оформление и написание отчёта о выполнении задания по производственной практике.

1 ОПИСАНИЕ ПРЕДПРИЯТИЯ

1.1 Структура предприятия и описание его бизнес-процессов

Modsen, основанная в 2020 году, представляет собой динамично развивающегося международного IT-провайдера, который за короткий срок зарекомендовал себя как надёжный партнёр в области цифровой трансформации. Компания предлагает полный спектр IT-услуг, охватывающий разработку программного обеспечения на заказ, усиление команд разработчиков, обеспечение качества, внедрение решений на базе искусственного интеллекта и машинного обучения, облачные технологии и консультационные услуги. Modsen работает с клиентами по всему миру, включая крупные корпорации и перспективные стартапы, адаптируясь к их потребностям и оперативно реагируя на изменения требований.

Организационная структура компании выстроена для максимальной эффективности. Руководство, включающее генерального директора, технического директора и глав ключевых направлений — маркетинга, стратегического развития, продаж, DevOps и HR, — задаёт курс развития и координирует деятельность подразделений. Техническую основу составляет команда из около 70 разработчиков, среди которых специалисты по .NET, Java, full-stack, frontend и мобильной разработке, применяющие гибкие методологии, такие как Agile и Scrum, для реализации проектов.

Отдел IT-решений, насчитывающий примерно 22 эксперта, отвечает за консультирование, управление проектами, анализ требований, подготовку документации и контроль выполнения задач. Маркетинговый отдел, в котором работает около 51 сотрудника, вместе с командой продаж и клиентского успеха, состоящей из примерно 10 человек, обеспечивает привлечение новых проектов и поддержание долгосрочных отношений с клиентами. HR-отдел играет важную роль в подборе и адаптации персонала, создании комфортной рабочей среды и укреплении корпоративной культуры.

Такая структура позволяет Modsen оперативно формировать команды под конкретные задачи и эффективно реализовывать модель расширения команд, интегрируя своих специалистов в проекты заказчиков на временной или постоянной основе. Гибкость и клиентоориентированность помогают компании успешно справляться с задачами любой сложности, поддерживая высокий уровень качества и удовлетворённости клиентов.

1.2 Организация экономической работы

Для обеспечения планово-экономической деятельности в ООО «Модсен» функционирует управление экономической, юридической и кадровой работы. Руководство этим подразделением осуществляет начальник управления, который назначается на должность приказом директора предприятия. Состав управления определяется директором с учётом особенностей деятельности компании на основе предложений,

представленных начальником управления.

Основные задачи управления включают разработку единой экономической политики предприятия, основанной на анализе текущего состояния и тенденций развития IT-отрасли. Управление занимается планированием расходов, необходимых для деятельности компании, а также разработкой инвестиционных программ. Важной частью работы является формирование экономической стратегии, направленной на адаптацию хозяйственной деятельности и системы управления к изменяющимся рыночным условиям, как внешним, так и внутренним.

Управление координирует подготовку проектов текущих планов всеми подразделениями компании, обеспечивая их соответствие заказам клиентов и условиям заключённых договоров. Оно выявляет продукцию или услуги, приносящие убытки, и разрабатывает меры по их исключению из деятельности. Также управление проводит комплексный анализ всех аспектов работы предприятия, включая производственные и технико-экономические показатели, ведёт статистический учёт, систематизирует данные и подготавливает статистическую отчётность в установленные сроки.

Для повышения эффективности работы управление активно взаимодействует с другими подразделениями компании, обеспечивая прозрачность финансовых процессов и своевременное выполнение задач. Это позволяет оперативно реагировать на изменения в бизнес-среде и корректировать планы в соответствии с новыми приоритетами клиентов или рынка.

Важной функцией управления является поддержание юридической и кадровой стабильности. Оно обеспечивает соответствие деятельности компании законодательным требованиям, а также занимается вопросами подбора, обучения и мотивации персонала, что способствует созданию устойчивой и продуктивной рабочей среды в ООО «Модсен».

1.3 Индивидуальное задание

После начала практики и согласования с руководителем были определены следующие индивидуальные задачи:

- выполнение тестового задания, направленного на закрепление и демонстрацию полученных знаний;
- регулярное решение алгоритмических задач на платформе CodeWars для развития навыков программирования и повышения уровня владения языком;
- прохождение теоретического курса с последующим выполнением тестовых и практических заданий для закрепления изученного материала и проверки понимания ключевых тем.

2 ОБЗОР ЛИТЕРАТУРЫ

2.1 Java

Java – это объектно-ориентированный язык программирования, который занимает лидирующие позиции в индустрии разработки программного обеспечения благодаря своей универсальности, надёжности и богатому набору инструментов. Он широко применяется для создания корпоративных, мобильных, десктопных и веб-приложений, а также для разработки микросервисных архитектур и облачных решений. Одной из ключевых особенностей Java является принцип «write once, run anywhere» («написал однажды – запускай везде»), который реализуется посредством использования виртуальной машины Java (JVM). Благодаря этому программы, написанные на Java, компилируются в промежуточный байт-код, который может быть выполнен на любой платформе, оснащённой соответствующей виртуальной машиной, что обеспечивает истинную кроссплатформенность и независимость от операционной системы.

Java поддерживает основные принципы объектно-ориентированного программирования (ООП), такие как инкапсуляция, наследование и полиморфизм, что позволяет создавать гибкие, масштабируемые и легко сопровождаемые программные решения. Инкапсуляция способствует сокрытию внутренней реализации классов и упрощает управление сложностью кода, наследование облегчает повторное использование и расширение функциональности, а полиморфизм обеспечивает универсальность работы с объектами различных типов. В языке реализованы мощные средства для создания иерархий классов, интерфейсов, абстракций, что особенно важно при проектировании архитектуры крупных программных систем.

В ходе производственной практики Java выступила в качестве основного инструмента для реализации серверной части приложения, отвечающей за обработку бизнес-логики, взаимодействие с базой данных и обеспечение надёжности всей системы. Были разработаны ключевые сущности (entity), контроллеры для обработки HTTP-запросов, сервисные классы, отвечающие за выполнение бизнес-операций, а также репозитории для абстракции доступа к данным. Архитектура приложения строилась на основе принципа разделения ответственности (controller-service-repository), что повысило масштабируемость, облегчило тестирование и сопровождение проекта.

Для взаимодействия с реляционной базой данных активно использовались технологии объектно-реляционного отображения (ORM), в частности – фреймворк Hibernate, реализующий спецификацию JPA (Java Persistence API). Это дало возможность работать с данными на уровне объектов, минимизировать написание ручных SQL-запросов, а также воспользоваться преимуществами автоматического управления транзакциями, кэширования и генерации схемы базы данных. Репозитории позволяли реализовать как стандартные CRUD-операции, так и сложные

пользовательские запросы, что обеспечило гибкость и производительность при работе с большими объёмами информации.

В проекте были применены современные возможности языка Java: лямбда-выражения, позволяющие писать компактный и выразительный код для обработки коллекций; Stream API, предоставляющий мощные средства для функционального программирования и параллельной обработки данных; аннотации для внедрения зависимостей, валидации данных и конфигурирования компонентов; обобщения (generics), обеспечивающие типовую безопасность и универсальность кода. Всё это способствовало созданию надёжной, расширяемой и легко сопровождаемой архитектуры приложения.

Важной частью практики стало внедрение встроенных механизмов обработки исключительных ситуаций (exceptions), что позволило повысить устойчивость приложения к ошибкам и обеспечить информативную обратную связь для пользователей. Были реализованы централизованные обработчики ошибок, собственные классы исключений, а также механизмы логирования, что упростило диагностику проблем и сопровождение системы. Кроме того, применялись лучшие практики по организации структуры проекта, написанию читаемого и поддерживаемого кода, что соответствует требованиям современной промышленной разработки программного обеспечения на языке Java.

2.2 PostgreSQL

PostgreSQL – это одна из самых популярных и надёжных систем управления реляционными базами данных с открытым исходным кодом, широко применяемая в индустрии разработки программного обеспечения благодаря своей высокой производительности, надёжности и гибкости. Она обеспечивает полноценную поддержку стандартов SQL, а также обладает богатым набором расширяемых возможностей, что делает её востребованной как в масштабных корпоративных системах, так и в проектах малого и среднего бизнеса, стартапах и научных исследованиях. Одной из ключевых особенностей PostgreSQL является строгая поддержка стандартов ACID (атомарность, согласованность, изолированность, устойчивость), что гарантирует целостность, достоверность и надёжную защиту данных при выполнении любых операций. Система активно поддерживает расширяемость: пользователи могут определять собственные типы данных, функции, операторные классы, а также подключать внешние расширения для реализации специфических бизнес-задач.

PostgreSQL поддерживает работу с самыми разными типами данных – помимо стандартных числовых, символьных и дат, это массивы, JSON, XML, геоданные (PostGIS), а также сложные пользовательские структуры. Система предоставляет расширенные средства работы с транзакциями, включая уровни изоляции вплоть до Serializable, что особенно важно для критичных к согласованности данных приложений. Среди других возможностей стоит

отметить поддержку внешних ключей, триггеров, представлений, полнотекстового поиска, оконных функций, рекурсивных запросов и хранимых процедур, что сильно расширяет сферы применения PostgreSQL. Для работы с semi-structured данными и интеграции с современными веб-сервисами реализована полноценная поддержка JSON и JSONB, позволяющая эффективно хранить и обрабатывать неструктурированные данные.

В рамках производственной практики PostgreSQL использовалась в качестве основной системы хранения данных для серверного приложения, реализующего бизнес-логику, учёт транзакций, а также хранение информации о пользователях, уведомлениях и других сущностях предметной области. Вся работа с базой данных строилась на использовании ORM-фреймворка Hibernate (JPA), что позволило работать с данными на уровне объектной модели, упростить внедрение изменений и повысить надёжность работы с транзакциями. Были спроектированы и реализованы таблицы для всех ключевых сущностей приложения, определены связи между ними (one-to-many, many-to-one, many-to-many), а также настроены каскадные действия для автоматического обновления и удаления связанных записей.

Особое внимание уделялось обеспечению целостности и согласованности данных – для этого применялись ограничения уникальности, not null, проверки внешних ключей, а также механизмы каскадных обновлений и удалений. Для повышения производительности были созданы индексы на наиболее часто используемые поля, что существенно ускорило выполнение сложных выборок, фильтраций и сортировок данных. В процессе разработки структуры и схемы базы данных активно применялись миграционные инструменты, такие как Liquibase (или Flyway), которые позволяли формировать версионные SQL-скрипты, отслеживать все изменения схемы, обеспечивать автоматическое и воспроизводимое развертывание на разных средах, а также минимизировать риски потери данных при обновлении приложения.

Для взаимодействия с базой данных в ходе практики активно использовались различные виды SQL-запросов: простые выборки, сложные объединения таблиц (join), агрегации, группировки, подзапросы, оконные функции и рекурсивные запросы. Особое внимание уделялось анализу и оптимизации производительности – изучались планы выполнения запросов, подбирались оптимальные индексы, реорганизовывались сложные выборки для снижения нагрузки на сервер. Такой подход позволил обеспечить высокую скорость обработки данных даже при больших объёмах информации и большом количестве одновременных пользователей.

Система PostgreSQL показала себя как стабильная, надёжная и высокопроизводительная СУБД, способная обслуживать как частые короткие транзакции, так и сложные аналитические запросы в реальном времени. Для обеспечения безопасности данных была организована система ролей и прав доступа: для каждой среды (разработка, тестирование, продакшн) создавались отдельные пользователи с минимально необходимыми привилегиями, что позволило ограничить доступ к критически важной информации и

предотвратить несанкционированные действия. Кроме того, была налажена регулярная система резервного копирования базы данных и тестирование восстановления из бэкапов, что обеспечило дополнительную защиту от потери данных и повысило надёжность работы приложения в целом.

2.3 Spring Framework

Spring Framework – это широко распространённый фреймворк для разработки корпоративных приложений на языке Java, который позволяет создавать масштабируемые, надёжные и поддерживаемые программные решения. Основное преимущество Spring заключается в его модульности и гибкости: разработчик может использовать только необходимые компоненты, такие как Spring Core, Spring Data, Spring MVC, Spring Security и другие, что позволяет эффективно строить архитектуру приложения и минимизировать избыточность кода. Одной из ключевых особенностей Spring является внедрение зависимостей (Dependency Injection), что упрощает управление объектами и их связями, делает код более тестируемым и облегчает сопровождение больших проектов.

В рамках практики Spring Framework применялся для создания серверной части приложения, отвечающей за обработку бизнес-логики, взаимодействие с базой данных и реализацию REST API. Для построения API использовался модуль Spring Web (Spring MVC), который обеспечивает маршрутизацию запросов, работу с параметрами, сериализацию и десериализацию данных, а также обработку ошибок на уровне контроллеров. Благодаря аннотациям, таким как `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping` и другим, структура кода становится максимально лаконичной и понятной, что облегчает чтение, поддержку и развитие приложения.

Для работы с данными использовался модуль Spring Data JPA, который позволил легко интегрировать приложение с реляционной базой данных PostgreSQL через ORM-фреймворк Hibernate. Использование репозиторий (интерфейсов, расширяющих `JpaRepository` или `CrudRepository`) дало возможность быстро реализовывать операции сохранения, поиска, обновления и удаления сущностей без написания большого количества шаблонного кода. Spring автоматически реализует основные методы репозиторий, а для сложных запросов можно определять методы с помощью специальной нотации в их именах или использовать язык запросов JPQL.

Безопасность приложения обеспечивалась с помощью модуля Spring Security, позволяющего реализовывать аутентификацию и авторизацию пользователей, управлять ролями и правами доступа, а также защищать эндпоинты приложения от несанкционированного доступа. Конфигурация безопасности может осуществляться как на уровне конфигурационных файлов, так и с использованием аннотаций и Java-конфигурации, что даёт высокую гибкость и масштабируемость.

В процессе разработки активно использовались механизмы обработки

ошибок и валидации входных данных, предоставляемые Spring. Для валидации применялись аннотации, такие как `@Valid` и `@NotNull`, что позволило автоматически проверять корректность входящих данных и возвращать пользователю понятные сообщения об ошибках. Для централизованной обработки исключений применялись классы с аннотацией `@ControllerAdvice`, где настраивалась логика реагирования на различные типы ошибок.

Использование Spring Framework значительно ускорило процесс разработки, позволило соблюдать лучшие практики проектирования и обеспечило высокое качество и надёжность итогового программного продукта. Благодаря широкому сообществу и обилию документации интеграция новых технологий и библиотек происходила быстро и без затруднений, а структура проекта осталась прозрачной и удобной для дальнейшего расширения и поддержки.

2.4 Maven

Maven – это один из самых популярных и мощных инструментов для автоматизации сборки, управления зависимостями и организации жизненного цикла программных проектов в экосистеме Java. Его использование значительно повышает эффективность командной работы, стандартизирует процессы и облегчает сопровождение приложений на всех этапах развития. Основная концепция Maven заключается в декларативном описании структуры и параметров проекта с помощью файла `pom.xml`, где задаются используемые библиотеки, плагины, настройки сборки, профили окружений и другие важные аспекты. Такой подход позволяет разработчикам минимизировать рутинные трудозатраты, повысить однородность проектов и обеспечить воспроизводимость результатов во всех средах – от локальной разработки до промышленной эксплуатации.

Одним из главных достоинств Maven является централизованное управление зависимостями. Вместо ручного скачивания и подключения внешних библиотек разработчик указывает необходимые артефакты в `pom.xml`, а Maven самостоятельно загружает их из центральных или локальных репозиториев, разрешает конфликты версий и следит за целостностью графа зависимостей. Это существенно ускоряет внедрение новых технологий, обновление используемых компонентов и снижает риск появления несовместимостей между библиотеками. Дополнительно, через систему плагинов Maven можно добавить автоматическую генерацию документации, анализ качества кода, статический анализ, сборку отчётов, интеграцию с системами CI/CD и многое другое.

Maven реализует чёткую модель жизненного цикла проекта, включающую последовательность фаз: `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`. Каждая фаза отвечает за определённый этап обработки проекта: от проверки корректности структуры и компиляции исходного кода до запуска тестов, упаковки приложения, публикации артефактов и деплоя на серверы.

Такой подход позволяет полностью автоматизировать процесс сборки и развертывания, минимизировать влияние человеческого фактора и ускорить выпуск новых версий продукта. Профили окружений дают возможность адаптировать сборку под различные условия (development, test, production), а шаблоны archetype – быстро создавать типовые структуры проектов с заготовленными настройками.

В рамках производственной практики Maven активно использовался для управления зависимостями серверного приложения, автоматизации сборки, запуска тестов и интеграции с внешними инструментами. В `pom.xml` были подробно описаны все используемые библиотеки, версии фреймворков, плагины для компиляции, генерации документации, запуска тестов (JUnit, Mockito), анализа покрытия кода, а также настройки для интеграции с Docker, Liquibase и средствами безопасности (например, Spring Security). Это обеспечило единый стандарт для всей команды, позволило быстро подключать к проекту новых участников и гарантировало идентичность окружения вне зависимости от операционной системы или локальных настроек разработчиков.

Дополнительным преимуществом использования Maven стала его интеграция с инструментами непрерывной интеграции и доставки (Jenkins, GitHub Actions и т.д.). Возможность автоматически запускать сборку, тестирование и деплой после каждого коммита ускорила цикл обратной связи и повысила надёжность релизного процесса. В рамках практики был освоен опыт настройки multi-module проектов, когда несколько взаимосвязанных модулей (например, основной сервис, база данных, интеграционные тесты) управляются единым `pom.xml` на уровне родительского проекта и могут развиваться независимо друг от друга.

Особое внимание уделялось поддержанию чистоты и актуальности файла `pom.xml`, своевременному обновлению зависимостей и анализу транзитивных библиотек на предмет уязвимостей или несовместимостей. Были изучены возможности настройки профилей для различных сценариев сборки, работы с приватными и корпоративными репозиториями, настройки проху и управления лицензиями используемых компонентов. Такой подход обеспечил высокий уровень безопасности, прозрачности и предсказуемости всего процесса разработки и сопровождения приложения.

2.5 Инструменты разработки: IntelliJ IDEA

IntelliJ IDEA – это современная интегрированная среда разработки (IDE), широко используемая для создания приложений на языке Java и многих других технологиях. Она отличается богатым функционалом, высокой производительностью и развитой экосистемой плагинов, что делает её одним из самых востребованных инструментов в профессиональной среде разработчиков. В рамках производственной практики IntelliJ IDEA использовалась не только для написания и редактирования кода, но и для комплексной поддержки всего жизненного цикла разработки программного

обеспечения – от проектирования архитектуры до тестирования, отладки и деплоя.

Одним из ключевых преимуществ IntelliJ IDEA является интеллектуальная система автодополнения и анализа кода, которая позволяет ускорить процесс написания программ, снизить вероятность ошибок и повысить читаемость. IDE автоматически распознаёт структуры кода, предлагает подсказки по использованию классов, методов и переменных, а также помогает быстро перемещаться между различными частями большого проекта благодаря мощной системе поиска, навигации и визуализации структуры. Инструменты рефакторинга позволяют безопасно и быстро изменять архитектуру проекта, переименовывать элементы, извлекать методы и классы, а также применять другие преобразования без риска нарушить работоспособность приложения.

Для работы с базой данных в IntelliJ IDEA встроены инструменты, позволяющие подключаться к различным СУБД, просматривать и редактировать структуру таблиц, выполнять SQL-запросы, анализировать результаты и отслеживать изменения схемы базы данных. В рамках практики это значительно упростило интеграцию серверного приложения с PostgreSQL, позволило быстро выявлять и устранять ошибки в логике взаимодействия с данными, а также проводить ручное тестирование запросов и миграций.

IntelliJ IDEA поддерживает интеграцию со множеством фреймворков и библиотек, включая Spring, Hibernate, Docker и другие инструменты, используемые в проекте. Для работы со Spring Framework IDE автоматически распознаёт конфигурационные файлы, предоставляет специализированные шаблоны кода, облегчает навигацию между компонентами и помогает выявлять потенциальные ошибки на ранних этапах разработки. Для тестирования бизнес-логики и других компонентов проекта использовалась интеграция с JUnit и Mockito, что позволило запускать и анализировать тесты прямо из среды разработки, получать отчёты о покрытии кода и своевременно выявлять неисправности.

Отдельное внимание уделялось автоматизации рутинных процессов с помощью встроённых инструментов для сборки проектов (Maven, Gradle), генерации документации, настройки окружения и работы с удалёнными серверами. IntelliJ IDEA также позволяет работать с Docker непосредственно из интерфейса IDE: запускать и останавливать контейнеры, просматривать логи, управлять образами и осуществлять отладку приложений, работающих в контейнерах.

2.6 Инструменты аутентификации: JWT (JSON Web Token)

JWT (JSON Web Token) – это современный, гибкий и широко распространённый стандарт передачи данных между участниками системы в защищённом и компактном виде. Основное назначение JWT – организация безопасной аутентификации и авторизации пользователей, а также хранение информации о пользователях и их правах доступа в распределённых и

микросервисных архитектурах. Токен JWT представляет собой строку, состоящую из трёх частей – заголовка (header), полезной нагрузки (payload) и подписи (signature), каждая из которых кодируется методом Base64, что обеспечивает компактность и универсальность формата. В заголовке содержится информация о типе токена и алгоритме подписи, в полезной нагрузке – любые необходимые данные (например, идентификатор пользователя, его роль, срок действия токена, дополнительные атрибуты и метаданные), а подпись гарантирует целостность и неизменность токена при передаче между клиентом и сервером.

В рамках производственной практики JWT применялся для реализации механизма аутентификации и авторизации пользователей при работе с REST API. После успешного прохождения процедуры входа на сервере генерировался уникальный токен JWT, который возвращался клиентскому приложению и сохранялся на стороне клиента (например, в localStorage браузера или в памяти мобильного приложения). При каждом последующем запросе к защищённым ресурсам клиент передавал токен в заголовке Authorization, что позволяло серверу быстро и безопасно идентифицировать пользователя, не обращаясь к базе данных при каждом запросе. Такой подход обеспечивает stateless-архитектуру, при которой серверу не требуется хранить сессии и дополнительную информацию о пользователях, а все необходимые данные содержатся непосредственно в токене.

Для генерации, подписи и проверки JWT в проекте использовались специализированные библиотеки, интегрированные с Spring Security, что позволяло автоматизировать процесс создания, передачи и валидации токенов. При генерации токена в него включались уникальный идентификатор пользователя, срок действия (expiration), роль, а также дополнительная служебная информация для контроля доступа. Подпись токена осуществлялась с использованием секретного ключа, который хранился на сервере и был недоступен для внешних пользователей. При получении запроса от клиента сервер проверял подпись токена, сравнивал срок его действия и извлекал необходимые данные для разграничения прав доступа к ресурсам.

2.7 Инструменты управления версиями: Git

Git – современная распределённая система управления версиями, которая стала стандартом де-факто в профессиональной разработке программного обеспечения. Её основное предназначение заключается в отслеживании любых изменений в коде, обеспечении возможности быстрого возврата к предыдущим версиям, а также в эффективной поддержке командной работы над проектами любой сложности и масштаба. Применение Git в рамках производственной практики охватывало все этапы разработки: от первоначальной инициализации репозитория до релизов, сопровождения и поддержки программного продукта.

3 ВЫПОЛНЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

3.1 Архитектура и организация проекта

В процессе производственной практики был реализован серверный программный продукт для интернет-магазина – InternetShopAPI. Проект разрабатывался с использованием стека современных технологий на языке Java, в основе лежала платформа Spring Boot (версия 3.5.0), а основная архитектура строилась по принципам RESTful-сервисов. Такой подход обеспечил гибкость, масштабируемость и удобство сопровождения. В качестве системы управления зависимостями и сборки проекта использовался Maven, что позволило централизованно управлять версиями библиотек, плагинов, а также интеграцией различных инструментов и фреймворков в единое целое.

Структура проекта включала несколько основных модулей: слой контроллеров (REST API), сервисный слой (бизнес-логика), слой доступа к данным (репозитории на базе Spring Data JPA), а также слой безопасности (Spring Security с поддержкой JWT-аутентификации). Для хранения и обработки данных применялась реляционная база данных PostgreSQL, а для миграций структуры – инструмент Liquibase, позволяющий автоматически применять изменения схемы базы при каждом деплое.

В проекте активно использовались аннотации Lombok для сокращения шаблонного кода (геттеры, сеттеры, конструкторы и т. д.), что повысило читаемость и упростило сопровождение программы. Для преобразования данных между слоями (например, между сущностями и DTO) применялся MapStruct, обеспечивающий автоматическую генерацию мапперов и сокращение риска ошибок при ручном копировании данных.

3.2 Работа с базой данных и миграциями

Одной из ключевых задач на этапе разработки стало проектирование и реализация структуры базы данных. Для этого была выбрана PostgreSQL – современная, надёжная и масштабируемая СУБД с поддержкой транзакций, индексов, сложных запросов и расширяемости. Для управления версионностью базы данных и автоматизации процесса её обновления использовался Liquibase. Все изменения (создание таблиц, добавление полей, индексов, ограничений) оформлялись в виде changelog-файлов, которые автоматически применялись при запуске приложения.

Это обеспечило полную синхронизацию структуры базы данных между всеми средами (разработка, тестирование, продакшн) и позволило избежать ошибок, связанных с ручным обновлением схемы. В проекте была реализована система миграций, что позволяло быстро реагировать на изменения требований и добавлять новые сущности или атрибуты без риска нарушения целостности данных. Такой подход способствовал стабильной и безопасной работе приложения, а также упростил коллективную работу над проектом.

Взаимодействие с базой данных осуществлялось с помощью Spring Data JPA. Репозитории позволяли использовать как стандартные CRUD-операции, так и собственные методы поиска по критериям, что обеспечивало гибкость при реализации бизнес-логики. Для тестирования корректности работы запросов активно использовались интеграционные тесты, а также возможности визуализации и ручного исполнения SQL-запросов через инструменты IDE (IntelliJ IDEA) и внешние клиенты.

3.3 Разработка REST API и бизнес-логики

В рамках практики основное внимание было сосредоточено на создании чистой и масштабируемой архитектуры REST API. Контроллеры обрабатывали входящие HTTP-запросы, проверяли корректность данных с использованием аннотаций Spring Validation, передавали управление соответствующим сервисным методам и формировали структурированные ответы в формате JSON для клиента. Для документирования API применялся springdoc-openapi, что позволило автоматически генерировать интерактивную документацию через Swagger UI, охватывающую все доступные эндпоинты.

Бизнес-логика была реализована в отдельных сервисных классах, что обеспечило чёткое разделение ответственности, упростило тестирование и сделало систему более гибкой для будущих доработок. Для автоматизации преобразования данных между сущностями и DTO использовался MapStruct, устранивший необходимость ручного копирования полей и снизивший риск ошибок при рефакторинге. Особое внимание уделялось обработке исключений, созданию унифицированных сообщений об ошибках для клиента и логированию ключевых событий, что повысило надёжность системы и облегчило её сопровождение.

В процессе работы над API также изучались методы оптимизации производительности и обеспечения стабильности системы. Были рассмотрены подходы к кэшированию данных, настройке пагинации и оптимизации запросов к базе данных, что позволило повысить скорость обработки запросов и снизить нагрузку на сервер. Эти навыки стали важным дополнением к пониманию разработки современных веб-приложений.

3.4 Обеспечение безопасности: Spring Security и JWT

Для защиты API и управления доступом в проекте был реализован модуль безопасности на базе Spring Security с поддержкой JWT (JSON Web Token). После успешной аутентификации пользователя сервер генерировал JWT-токен, который возвращался клиенту и использовался для последующих запросов к защищённым эндпоинтам. Это обеспечило stateless-модель взаимодействия: сервер не хранил сессии пользователей, а вся необходимая информация (идентификатор, роль, срок действия) содержалась в самом токене.

Валидация JWT и разграничение прав доступа реализовывалось с

помощью фильтров Spring Security, интегрированных с библиотекой jjwt. Такой подход позволил гибко управлять ролями пользователей (например, администратор, обычный пользователь) и защищать критически важные ресурсы от несанкционированного доступа. Для тестирования работы механизма безопасности использовались как автоматизированные тесты на стороне серверной логики, так и ручные проверки с помощью Postman, что позволило выявить и устранить потенциальные уязвимости.

3.5 Среда разработки и инструменты автоматизации

Вся разработка велась в среде IntelliJ IDEA, которая предоставляла мощные средства для написания, рефакторинга и отладки кода, работы с базой данных, управления зависимостями и интеграции с системами контроля версий (Git). Благодаря поддержке необходимых фреймворков и расширений IDE облегчала навигацию по проекту, автоматическую генерацию кода (например, через Lombok и MapStruct), а также управление Docker-контейнерами непосредственно из интерфейса.

Git использовался как основная система контроля версий. Для каждой задачи создавалась отдельная ветка, после чего изменения проходили ревью через pull request. Такой подход позволял отслеживать историю изменений, быстро выявлять и устранять ошибки, а также координировать работу между всеми участниками команды. Правильная организация ветвления (feature-ветки, bugfix, hotfix) и продуманное оформление коммитов обеспечили прозрачность и надёжность процесса разработки.

3.6 Документирование и поддержка API

Для эффективного взаимодействия с внешними клиентами и другими разработчиками в проекте применялась автоматическая генерация документации REST API с помощью springdoc-openapi. Это позволило не только описывать все доступные эндпоинты, параметры запросов и ответы, но и предоставлять интерактивный интерфейс для ручного тестирования API через Swagger UI. Такой подход ускорил интеграцию внешних систем, повысил прозрачность архитектуры и упростил сопровождение продукта.

ЗАКЛЮЧЕНИЕ

Прохождение производственной практики позволило глубоко погрузиться в структуру работы предприятия, изучить его нормативную базу и должностные инструкции сотрудников. В процессе работы были освоены практические навыки в области информатики и разработки программного обеспечения, что стало важным шагом в профессиональном становлении. Для выполнения индивидуального задания изучалась специализированная литература, а также применялся опыт, накопленный за время практики, что обеспечило успешное выполнение практической части работы.

В ходе практики была детально изучена организационная структура предприятия, что дало понимание, как взаимодействуют различные подразделения для достижения общих целей. Погружение в нормативную документацию и должностные обязанности сотрудников позволило разобраться в стандартах работы компании и используемых программно-технических средствах. Особое внимание уделялось анализу технологических и бизнес-процессов, реализуемых на предприятии, что помогло понять их роль в деятельности компании.

Практика включала исследование конкретного программного продукта в рамках индивидуального задания. Это позволило не только углубить знания в области разработки, но и выявить ключевые аспекты, влияющие на качество программного обеспечения. На основе технической задачи, поставленной в ходе практики, был сформирован перечень требований, необходимых для её реализации. Также был собран материал, который станет основой для технической задачи в рамках будущего дипломного проекта.

Работа на практике способствовала развитию аналитических навыков и умения систематизировать информацию. Анализ программного продукта выявил сильные и слабые стороны, что позволило предложить рекомендации по его улучшению. Это стало важным опытом в понимании, как теоретические знания применяются для решения реальных задач.

Практика также дала возможность освоить методы работы с технической документацией и взаимодействия с командой разработчиков. Участие в реальных проектах предприятия помогло понять, как выстраивается процесс от постановки задачи до её реализации. Этот опыт укрепил уверенность в профессиональных навыках и подготовил к будущей работе над дипломным проектом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Введение в Spring Boot [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/435144/>.
- [2] PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql/current/>.
- [3] Spring Documentation [Электронный ресурс]. – Режим доступа: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle>.
- [4] Liquibase Documentation [Электронный ресурс]. – Режим доступа: <https://docs.liquibase.com/>.
- [5] MapStruct Documentation [Электронный ресурс]. – Режим доступа: <https://mapstruct.org/documentation/>.
- [6] JWT Introduction [Электронный ресурс]. – Режим доступа: <https://jwt.io/introduction/>.